

Project Presentation of Industry Oriented Hands on Experience (22CS422)

On

SketchSQL

Saksham Dogra

2210990763

Supervised By

Dr. Monika Aggarwal

Department of Computer Science and Engineering,
Chitkara University, Punjab

- SketchSQL is a visual database schema designer with an integrated AI assistant
- Tagline: “Draw your database. Get your SQL. Ask AI anything about it.”
- Built as a Final Year CSE Industry Oriented Project (22CS422) at Chitkara University
- Target Users: Backend developers, full-stack engineers, DB students, DBAs, and technical writers
- Core Value Loop:
 - Design ER diagrams visually on a drag-drop canvas
 - Auto-generate MySQL / PostgreSQL DDL in real time
 - Export ORM models (Django, Prisma, SQLAlchemy)
 - AI assistant auto-builds schemas from plain English

- Problem Statement:
 - Designing relational schemas is error-prone and time-consuming when done manually
 - Developers frequently switch between diagramming tools and SQL editors — no unified environment
 - AI assistance for schema generation is largely absent in existing tools
- Objectives:
 - Build a single-page tool that combines ER diagramming + SQL DDL generation
 - Integrate AI to auto-generate schemas from natural language descriptions
 - Support ORM code generation for popular frameworks
 - Keep the app fully client-side and offline-capable for core features
 - Provide a shareable read-only public link for portfolio/handoff use

- Frontend:
 - React 19 (CRA + Craco), ReactFlow 11 — canvas and custom node/edge types
 - Zustand 5 — global state management (diagram store + UI store)
 - Monaco Editor — SQL / ORM code viewer with syntax highlighting
 - Tailwind CSS + CSS custom properties — dark/light themed design system
- Backend:
 - Python 3 + FastAPI (Uvicorn) — thin API server on port 8001
 - Azure OpenAI GPT-5.4 — AI schema generation, analysis, and chat
 - Motor (async MongoDB driver) — used only for public share snapshots
- Storage:
 - localStorage — autosave every 2 seconds
 - IndexedDB — named diagram saves (up to 50)
 - MongoDB — server-side share snapshots only

- Three-Panel Layout:
 - Left Panel — saved diagrams list, new diagram button, rename/delete
 - Center Canvas — ReactFlow ER diagram builder (drag-drop nodes and edges)
 - Right Panel — tabbed: SQL Output, ORM Output, AI Assistant, Properties
- Client-Server Separation:
 - All SQL generation and ORM generation runs client-side (zero latency)
 - Backend handles AI API calls and public share storage only
 - Frontend proxies `/api/*` to backend via `REACT_APP_BACKEND_URL`
- Routing (react-router-dom v7):
 - `/` — full editor (App.js with 3-panel layout)
 - `/share/:shareId` — read-only public view with Fork to Edit capability
- State Management — Zustand stores (diagramStore + uiStore) with 50-step undo/redo

Key Features — Canvas & SQL Generation



- Visual ER Diagram Builder:
 - Drag-drop tables; double-click canvas to add a new table
 - Custom TableNode: colored header, PK/UQ/NN badges, inline column editor
 - 16 SQL data types supported: INT, VARCHAR, TEXT, BOOLEAN, DATE, JSON, UUID, etc.
 - Relationship edges with cardinality labels: 1:1, 1:N, M:N
 - Snap-to-grid (20px), minimap, undo/redo (50 states), keyboard shortcuts
- SQL DDL Generation:
 - Client-side generation — instant, debounced at 800ms
 - Supports MySQL (AUTO_INCREMENT, backtick identifiers, ENGINE=InnoDB)
 - Supports PostgreSQL (SERIAL/BIGSERIAL, quoted identifiers, native BOOLEAN)
 - Topological sort ensures FK-referenced tables are created first
 - M:N relationships auto-emit junction table DDL with composite PK
 - Monaco editor output with copy and download (.sql) buttons

Key Features — ORM Generation & Import SQL

- ORM Code Generation (3 frameworks):
 - Django — models.py with ForeignKey, OneToOne, ManyToMany, class Meta: db_table
 - Prisma — schema.prisma with @id, @unique, @relation, @@map, back-references
 - SQLAlchemy — declarative Base, Column(), relationship(), ForeignKey with ondelete
 - Read-only Monaco editor, copy and download (.py or .prisma) per framework
- Import SQL — Reverse-Engineer Existing DDL:
 - Paste CREATE TABLE SQL (MySQL or PostgreSQL) into the import modal
 - Backend regex parser handles inline and table-level PKs, FKs, UNIQUE, NOT NULL
 - Supports backticks, double-quotes, brackets, IF NOT EXISTS, AUTO_INCREMENT, SERIAL
 - Imported tables auto-layouted on canvas using topological grid algorithm
- Export Options: SQL file, JSON diagram state, PNG, PDF (html-to-image + jsPDF)

AI Features (Azure OpenAI GPT-5.4)

- AI Feature 1 — Natural Language to Schema:
 - User describes a system in plain English (e.g. “E-commerce with products and orders”)
 - Backend sends prompt to Azure OpenAI GPT-5.4; response is strict JSON schema
 - Schema rendered as nodes/edges on canvas with auto-layout
 - Replace or merge with existing diagram (user choice modal)
- AI Feature 2 — Schema Analysis & Normalization Advice:
 - Sends full diagram JSON to GPT-5.4 for analysis
 - Returns: plain-English schema explanation, 1NF/2NF/3NF violations, missing indexes
 - Rendered as formatted markdown in the AI chat panel
- AI Feature 3 — Query Assistant (Multi-turn Chat):
 - Full conversation context with current schema JSON injected into every request
 - Writes complex JOINS, suggests indexes, answers schema questions
 - Typing indicator, markdown rendering, example prompt chips, clear-chat button

- Data Persistence (Local-First, No Auth):
 - Auto-save to localStorage every 2 seconds (key: sketchsql_autosave)
 - Named saves to IndexedDB (up to 50 diagrams) via left-panel list
 - No user accounts, no server-side storage for diagrams
- Public Share Feature:
 - Header Share button auto-POSTs diagram snapshot — returns 8-char URL-safe ID
 - Read-only /share/:id page: canvas, SQL/ORM tabs, view counter, READ-ONLY badge
 - Fork to Edit — clones diagram to autosave, navigates to editor
 - Snapshots stored in MongoDB shares collection; 404 for invalid IDs
- Backend API Routes (/api prefix):
 - POST /generate-sql — diagram JSON → SQL DDL (pure logic, no AI)
 - POST /import-sql — SQL string → diagram JSON (regex parser)
 - POST /ai/generate-schema, /ai/analyze-schema, /ai/chat — AI features
 - POST /share and GET /share/{shareId} — public link management

- Typical User Workflow:
 - Step 1: Open app — Welcome screen offers Start from Scratch, Load Example, or Describe with AI
 - Step 2: Add tables by double-clicking the canvas or using the toolbar
 - Step 3: Define columns (name, type, PK, FK, nullable, unique, default)
 - Step 4: Draw relationships by dragging column handles between tables
 - Step 5: SQL DDL auto-generates in the right panel (MySQL or PostgreSQL)
 - Step 6: Switch to ORM tab to get Django / Prisma / SQLAlchemy code
 - Step 7: Use AI Assistant to analyze, improve, or generate queries
 - Step 8: Save to IndexedDB or export as .sql / .json / .png / .pdf
- Built-in Example Schemas:
 - E-commerce: users, products, categories, orders, order_items, payments, addresses
 - Blog: users, posts, comments, tags, post_tags, likes
 - University: students, courses, instructors, enrollments, departments, grades
 - Hospital: patients, doctors, appointments, prescriptions, medications, departments

Application Workflow (Visual)

SketchSQL

**Visualize.
Design.
Generate.**

The modern way to design and generate database schemas with the power of AI.

- Visual ER Diagrams
- Multi-ORM Support
- AI-Powered Assistant
- One-Click Export

Schema Overview

University Management System

7 Tables
22 Columns
12+ Relationships

```
graph LR
    Students --> Departments
    Students --> Courses
    Students --> Enrollments
    Departments --> Instructors
    Departments --> Courses
    Instructors --> ClassSections
    Courses --> Semesters
    Semesters --> ClassSections
    ClassSections --> Enrollments
```

Key Features

- Visual ER Diagram Builder**
Drag, drop, and connect tables with an intuitive interface.
- Multi-ORM Support**
Generate models for Django ORM, Prisma, and SQLAlchemy.
- AI-Powered Assistant**
Get schema suggestions, analysis, and improvements using AI.
- One-Click Export**
Export your schema and code in multiple formats (SQL, JSON, Prisma, etc.).

AI Assistant in Action

Describe your needs. Get intelligent suggestions.

GENERATE FROM DESCRIPTION

Describe a university database schema.

Export to Prisma | Bug with constraints

University Management | Hospital system

Analyze My Schema | Done

Please analyze my current database schema and suggest improvements.

1. What This Schema Models

This schema models a basic university management registration system. It spans students, academic departments, instructors, courses, semesters, course offerings as class sections, and student enrollments in those sections.

Export & Share

Export your diagram and code in multiple formats.

Export SQL (DDL) | Export JSON (Prisma) | Export Prisma | Export PDF

Save your work, collaborate, and bring your database designs to life effortlessly.

- Key Achievements:
 - Fully functional visual ER diagram builder with real-time SQL DDL generation
 - Dual-dialect support: MySQL and PostgreSQL with correct syntax differences
 - ORM generation for 3 frameworks — eliminates manual boilerplate writing
 - AI-powered schema generation from plain English using Azure OpenAI GPT-5.4
 - Public shareable links with read-only view and Fork-to-Edit capability
 - Fully local-first, no authentication required, all core features work offline
- Future Work:
 - User authentication and cloud-synced diagram storage (Google OAuth / JWT)
 - Real-time collaboration using Yjs CRDT + WebSocket transport
 - OpenGraph preview image generation for shared diagram links
 - Additional SQL dialects: SQLite, Microsoft SQL Server
 - Additional ORM targets: TypeORM, Hibernate
 - Version history and diagram diffing for schema change tracking

Thank You